601.433/633 Introduction to Algorithms **Fall 2026**
Homework #4 **Due:** October 15, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Hashing (50 points)

We say that H is a 2-*universal hash family* if $\mathbf{Pr}_{h\sim H}[h(x) = a \wedge h(y) = b] \leq 1/M^2$ for all $x, y \in U$ with $x \neq y$ and $a, b \in \{0, 1, \ldots, M-1\}$.

(a) (25 points) Prove that any 2-universal hash family is also a universal hash family.

To prove that H is a universal hash family, we must prove that $\mathbf{Pr}_{h\sim H}[h(x) = h(y)] \leq \frac{1}{M}$ for all $x, y \in U$ with $x \neq y$. To find when $h(x) = h(y)$, we can set both to an arbitrary value $a \in \{0, 1, \ldots, M-1\}$. The definition never states that $a \neq b$, so we can write this as:

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] = \mathbf{Pr}_{h\sim H}[h(x) = a \wedge h(y) = a] \leq 1/M^2$

However, this just gives the probability that $h(x)$ and $h(y)$ to a, when we want to find when x and y map to any value in $\{0, 1, \ldots, M-1\}$. Thus, we can write the above as a summation over every possible value of a:

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] = \sum_{a=0}^{M-1}(\mathbf{Pr}_{h\sim H}[h(x) = a \wedge h(y) = a])$

For any a within the bounds, $\mathbf{Pr}_{h\sim H}[h(x) = a \wedge h(y) = a] \leq \frac{1}{M^2}$. Thus, we can set up the lower bound for each individual value inside the summation:

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] = \sum_{a=0}^{M-1}(\mathbf{Pr}_{h\sim H}[h(x) = a \wedge h(y) = a]) \leq \sum_{a=0}^{M-1}(\frac{1}{M^2})$

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] \leq \sum_{a=0}^{M-1}(\frac{1}{M^2})$

Simplifying by summing over the M terms of the summation:

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] \leq \frac{M}{M^2}$

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] \leq \frac{1}{M}$

We have proven that the definition of a universal hash family applies to any 2-universal hash family. Thus, any 2-universal hash family is a universal hash family

(b) (25 points) Prove that for every u and b with $u > b \geq 1$ there is some universal hash family from U to $\{0, 1, \ldots, M-1\}$ (with $|U| = 2^u$ and $M = 2^b$) which is *not* a 2-universal hash family. Hint: think about the constructions from class and the textbook.

To prove that any given construction leads to a universal hash family, we must prove:

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] \leq \frac{1}{2^b}$ for all $x, y \in U$ with $x \neq y$.

To prove that it is not a 2-universal hash family, we must prove that:

$\mathbf{Pr}_{h\sim H}[h(x) = i \wedge h(y) = j] > \frac{1}{M^2}$ for some distinct $x, y \in U$.

We first consider the construction in **Theorem 10.3.5** in the lexture notes, which states:

$\mathbf{Pr}_{h\sim H}[h(x) = h(y)] \leq \frac{1}{M}$ for all $x \neq y \in \{0,1\}^u$

In this theorem, $M = 2^b$ and $|\{0,1\}^u| = 2^u$, so we can see that this represents a universal hash family for every u and b, as per the specs in the question.

However, we must now examine its 2 universality before concluding the construction in class fits this proof. Any hash function in this family is defined as $h(x) = Gx \mod 2$, where G is a uniformly chose $b \times u$ matrix. Thus, there are 2^{bu} possible matrices (and therefore 2^{bu} functions in the hash family). By the definition of the hash function, any output $h(x)$ is a b-row, 1 column matrix. We consider 2 distinct inputs, x and y, and two output possibilities i and j.

We consider $\mathbf{Pr}_G[Gx = i \wedge Gy = j] = \frac{\#\text{ of matrices where } Gx=i \wedge Gy=j}{2^{bu}}$. Beginning with a simpler constraint, we consider $\mathbf{Pr}_G[Gx = i]$. For this, we choose a nonzero x. We can break this down by digit, and determine that $Gx = i$ means $g_1 \cdot x = i_1 \mod 2$, $g_2 \cdot x = i_2 \mod 2$, $g_3 \cdot x = i_3 \mod 2$, ... , $g_b \cdot x = i_b \mod 2$. We consider a single row g_k, with u entries, so there are 2^u possible rows. Since $x \neq 0$, the number of rows g_k satisfying $g_k \cdot x = i_k$ is 2^{u-1}. Each of the b rows of i has 2^{u-1} possible matrices satisfying it, so by multiplication, there are $2^{b(u-1)}$. Therefore:

$\mathbf{Pr}_G[Gx = i] = \frac{2^{b(u-1)}}{2^{bu}} = \frac{1}{2^b} = \frac{1}{M}$

Now, we consider a y equal to 0. Since we consider $x \neq 0$, this satisfies $x \neq y$. Since $y = 0$, we know that $Gy = j = 0$ regardless of what matrix G is chosen. Therefore, $\mathbf{Pr}_G[Gy = j] = 1$.

Since $Gy = j$ is always true, the logic value $(Gx = i \wedge Gy = j) = (Gx = i \wedge \text{TRUE}) = (Gx = i)$. Therefore, we can compute the probability of $\mathbf{Pr}_G[Gx = i \wedge Gy = j]$ as follows:

$\mathbf{Pr}_G[Gx = i \wedge Gy = j] = \mathbf{Pr}_G[Gx = i] = \frac{1}{M} > \frac{1}{M^2}$ for all $M > 1$.

Since M is at minimum 2, we have found an x and y within this universal hash family for which:

$\mathbf{Pr}_{h\sim H}[h(x) = i \wedge h(y) = j] > \frac{1}{M^2}$.

Therefore, we have proven the existence of some hash family for all u, b where $u > b \geq 1$ which is universal but not 2-universal.

2 Union-Find (50 points)

In class we proved that if we use trees to represent disjoint sets and use both union-by-rank and path compression, then the amortized cost of any operation (Make-Set, Find, or Union) is only $O(\log^* n)$. In this question we'll analyze what happens if we change our data structure in two ways: we do not use path compression, and we do union-by-cardinality rather than union-by-rank.

Slightly more formally, we change our data structure as follows. As before, every node has three values: an element, a parent pointer, and a value. But the value isn't a rank, but is rather the number of nodes in the subtree rooted at it, i.e., the *cardinality* of the subtree. So the root of any tree stores the cardinality of the set represented by the tree. The basic operations work as follows:

- Make-Set(x) simply returns a single node with the element x, where the parent pointer points to itself and the initial cardinality is 1.

- Find(x) follows the parent pointer from x recursively until it ends up at the root (which it knows since only the root will have its parent pointer point to itself), and then it returns the element at this root.
- Union(x, y) first does Find(x) to find the root r_x of the tree containing x and Find(y) to find the root r_y of the tree containing y. If $(r_x \to \textit{cardinality}) \geq (r_y \to \textit{cardinality})$, then we set the parent of r_y to be r_x and set $(r_x \to \textit{cardinality}) = (r_x \to \textit{cardinality}) + (r_y \to \textit{cardinality})$. Similarly, if $(r_y \to \textit{cardinality}) > (r_x \to \textit{cardinality})$, then we set the parent of r_x to be r_y and set $(r_y \to \textit{cardinality}) = (r_x \to \textit{cardinality}) + (r_y \to \textit{cardinality})$.

(a) (25 points) Prove that the worst-case running time of any operation is $O(\log n)$, where n is the number of Make-Set operations (i.e., the number of elements). Hint: can you bound the depth/height of a tree by its cardinality?

We first prove that the depth of a tree is bounded by its cardinality by strong induction. We will show that for any tree T within Union-Find structure F, the depth d is less than or equal to $\log_2$ of cardinality c – that is, $d_T \leq \log_2(c_T)$. Importantly, we define depth to be the maximum path size from root to leaf of a tree.

Base case: After performing Make-Set(x), tree T with 1 node is created, meaning T has a depth of 0. Since c_T is set to 1, we can see that $d_T = 0 \leq \log_2(c_T) = 0$. Thus, the base case holds.

Inductive case: We assume that for all possible cardinalities 1-k of T, $d_T \leq c_T$. We will prove that a tree T with cardinality $k+1$ has a depth less than c_T. To create T with cardinality $k+1$, trees T and R, where $c_T \geq c_R$ and $c_T + c_R = k+1$, must have a union performed. We can see that the cardinality of c_T must be incremented by at least 1, as c_R has cardinality at least 1. By the inductive hypothesis, $d_T \leq \log_2(c_T)$ and $d_R \leq \log_2(c_R)$. The union operation will join the root of R to the root of T.

The cases for the resulting depth of T are as follows:

(1) $d_T = d_R$: $d_T = d_{T\text{prior}} + 1 = d_R + 1$. We know that $d_R \leq \log_2(c_R)$, so $d_R + 1 \leq \log_2(c_R) + 1 = \log_2(c_R) + \log_2(2) = \log_2(2c_R)$. We know that $c_T \geq 2c_R$, since $c_T = c_{T\text{prior}} + c_R$ and $c_{T\text{prior}} \geq c_R$, so c_T must be at least double c_R. Therefore, $d_T = d_{T\text{prior}} + 1 = d_R + 1 \leq \log_2(2c_R) \leq \log_2(c_T)$, so $d_T \leq \log_2(c_T)$, and this case holds.

(2) $d_T > d_R$: $d_T = d_{T\text{prior}}$. Since $d_{T\text{prior}} \leq \log_2(c_{T\text{prior}})$ and c_T increases during the union, then d_T must remain less than or equal to $\log_2(c_{T\text{prior}} + c_R)$. Thus, $d_T \leq \log_2(c_T)$, and this case holds.

(3) $d_T < d_R$: $d_T = d_R + 1$. We know that $d_R \leq \log_2(c_R)$, so $d_R + 1 \leq \log_2(c_R) + 1 = \log_2(c_R) + \log_2(2) = \log_2(2c_R)$. We know that $c_T \geq 2c_R$, since $c_T = c_{T\text{prior}} + c_R$ and $c_{T\text{prior}} \geq c_R$, so c_T must be at least double c_R. Therefore, $d_T = d_R + 1 \leq \log_2(2c_R) \leq \log_2(c_T)$, so $d_T \leq \log_2(c_T)$, and this case holds.

Therefore, in every case, the inductive case holds.

In proving the base and inductive case, we have proven that the depth of a tree is bounded by $\log_2$ of its cardinality.

Each individual set is initially created with a cardinality of 1. We will prove that the cardinality of a tree is equal to the number of nodes in the tree by strong induction.

Base case: A newly created set has 1 node, and a cardinality of 1. Thus, this case holds.

Inductive case: We assume that a tree with 1 to k nodes has a cardinality of 1 to k respectively. We prove that a tree with $k+1$ nodes has a cardinality of $k+1$. A tree with $k+1$ cardinality is created when 2 trees whose cardinalities sum to $k+1$ are joined. By the inductive hypothesis, we know that the trees being unioned have a number of nodes equal to their cardinality, so the number of nodes in both trees sums to $k+1$. The resulting tree has all of the nodes in both of the trees, or the sum of the number of nodes in each tree, $k+1$. Thus, since the new number of nodes in the tree and the new cardinality is both $k+1$, the cardinality and number of nodes remains equal, so the inductive step holds.

In proving the base and inductive case, we have proven that the cardinality of a tree is equal to the number of nodes in a tree.

We can clearly see that Make-Set(x) runs in $O(1)$ time, but we must prove the runtime of Find(x) and Union(x, y). Both of these operations involve traversing at least 1 tree from a node to the root. Therefore, the runtime of these operations is bounded at $O(d)$. However, we know that the depth of the tree is bounded by the cardinality, so it is strictly less than $\log_2(c)$. Thus, we can conclude that any operation is bounded by $O(\log c)$. However, we proved that $c = \#$number of nodes, and the maximum number of nodes is at most the number of sets that have been created, or n, so $c \leq n$ for any tree. Therefore, both Find(x) and Union(x, y) have a runtime of $O(\log n)$.

Thus, the total runtime of any single operation is $O(\log n)$.

(b) (25 points) We now want to provide a matching lower bound not just on the worst-case running time, but even on the amortized running time. So suppose that we first do n Make-Set operations (so there are n elements). Give a sequence of $O(n)$ Union operations which cumulatively take $\Omega(n \log n)$ time (implying that the amortized cost of an operation is $\Omega(\log n)$). Each union operation should be a union of two distinct sets, i.e., you should never call Union(x, y) on elements x and y that are in the same set.

We seek to find a sequence where, after n Make-Set operations, there are $O(n)$ insertion operations which take a total of $\Omega(n \log n)$ time, meaning the amortized cost per operation is $\Omega(\log n)$.

To begin, there are n distinct sets, each with 1 node and therefore a cardinality of 1 (as we proved above). The sequence of Union operations behaves in layers to eventually 1 tree of $\frac{n}{2}$ nodes, leaving $\frac{n}{2}$ sets of 1 node. We consider the $\frac{n}{2}$ trees being merged in the following. We first merge each tree (of one node) with another partner. This yields $\frac{n}{4}$ trees with 2 nodes. Then, we merge each of the 2-node trees with another, yielding $\frac{n}{8}$ trees, each with 4 nodes. This process continues until there is 1 set with $\frac{n}{2}$ nodes. We see that since the number of sets is halved after each round of Unions, so there must be $\log(\frac{n}{2})$, or $\log(n) - 1 = O(\log n)$ rounds of merges to yield 1 set with n nodes. From here, we merge the $\frac{n}{2}$ remaining trees of 1 node with the 1 tree of $\frac{n}{2}$ nodes.

To prove that this sequence has $O(n)$ operations each with $\Omega(\log n)$ amortized cost, we must first prove that there are $O(n)$ Union operations. We can do so by summing over the number of operations at each level. At the first level, there are $\frac{n}{4}$ Unions, at the second, there are $\frac{n}{8}$, and this repeats $\log n$ times. Therefore, we can represent this sum as follows:

$\sum_{i=1}^{\log n}(\frac{n}{2^{(i+1)}})$

As n is constant within this summation, we can pull it out to yield the geometric summation:

$n \cdot \sum_{i=1}^{\log n}(\frac{1}{2}^{(i+1)}) = n \cdot [\frac{1}{4} \cdot (\frac{1-\frac{1}{2}^{\log(n)+1}}{1-\frac{1}{2}})] = n \cdot [\frac{1}{4} \cdot (2 \cdot (1-2^{\log(n)+1(-1)}))] = n \cdot [\frac{1}{2} \cdot (1-(2^{\log(n)(-1)} \cdot 2^{1(-1)}))] = n \cdot [\frac{1}{2} \cdot (1-(n^{-1} \cdot \frac{1}{2}))] = \frac{n}{2} \cdot (1-\frac{1}{2n})$

We can see that the term $1-\frac{1}{2n}$ grows closer to 1 as n grows, but it always remains strictly smaller than 1, so $\frac{n}{2} \cdot (1-\frac{1}{2n})$ grows very close to $\frac{n}{2}$ but never surpasses it, leading to an upper bound of $\frac{n}{2}$ operations.

From here, we perform $\frac{n}{2}$ Unions between the remaining $\frac{n}{2}$ trees of 1 node with the just-created tree of $\frac{n}{2}$ nodes. This string of operations involves a set of at most $\frac{n}{2}$ operations, followed by $\frac{n}{2}$ operations, so it has at most $\frac{n}{2}+\frac{n}{2}$ operations, or $O(n)$ operations

Now, we prove an overall cost of $\Omega(n \log n)$ for these n operations, meaning an amortized cost of $\Omega(\log n)$ per operation. We consider the second half of the operations in the sequence where there are $\frac{n}{2}$ Union operations of a tree of at least $\frac{n}{2}$ nodes with a tree of 1 node. We will call the tree of $\frac{n}{2}$ nodes B where l_B is a leaf node in B, and we will call an arbitrary 1-node tree S containing node s.

We know that l_B has depth $\log(\frac{n}{2})$. This is because this sequence unions trees of equal cardinality at every level. As per the specs of this question, when two trees of equal size are merged, the root of one tree becomes a child of the root of the other. Therefore, the height increases by one. After the first round, each tree has 2 nodes and height 1. After the second round, each tree has 4 nodes and height 2. After k rounds, each tree has 2^k nodes and height k. Following this pattern, tree B with $\frac{n}{2}$ nodes has height $\log(\frac{n}{2})$. Therefore a leaf node l_B has depth $\log(\frac{n}{2})$.

We consider the operation Union(s, l_B). To find the root r_B of B, the algorithm must traverse the height of B from l_B upwards. Since s is by default the root of S, finding the root takes $O(1)$ time. As l_B has depth $\log(\frac{n}{2})$, this operation must have $\Omega(\log(\frac{n}{2})) = \Omega(\log n)$ time. Since there are $\frac{n}{2}$ 1-node trees, this operation repeats $\frac{n}{2}$ times. Therefore, the complete runtime of these $\frac{n}{2}$ operations is $\Omega(\frac{n}{2} \log n)$, or $\Omega(n \log n)$.

In considering the second half of the operation, we have proven that this specific sequence of $O(n)$ Unions has a lower bound $\Omega(n \log n)$ time, meaning the amortized time per individual Union is $\Omega(\log n)$.